

Desarrollo de Servicios Web SOAP con GlassFish e IntelliJ

1. Conceptos Básicos

1.1. ¿Qué es Jakarta EE?

Jakarta EE es una plataforma que proporciona un conjunto de tecnologías y APIs para desarrollar aplicaciones web y empresariales en Java. Las aplicaciones empresariales son sistemas de software desarrollados para apoyar procesos y operaciones de organizaciones o empresas. **Jakarta EE** incluye tecnologías como Servlets, JSP, **JAX-WS**, **JAX-RS**, CDI y EJB para el desarrollo de aplicaciones empresariales en Java. Jakarta EE es la evolución de Java EE, actualmente mantenida por la Eclipse Foundation.

1.2. ¿Qué es un servidor de aplicaciones?

Un servidor de aplicaciones es una plataforma de software que permite desplegar, ejecutar y administrar aplicaciones empresariales, proporcionando servicios como manejo de peticiones, sesiones, seguridad, transacciones y acceso a bases de datos.

1.3. ¿Qué es GlassFish?

GlassFish es un servidor de aplicaciones Java que permite desplegar y ejecutar aplicaciones web empresariales desarrolladas con tecnologías Jakarta EE, como Servlets, JSP, JAX-WS y JAX-RS.

1.4. ¿Qué es un servicio web?

Un servicio web es una tecnología que permite que diferentes aplicaciones se comuniquen e intercambien información a través de Internet o una red.

1.5. ¿Qué es JAX-WS?

JAX-WS es una API de Java utilizada para crear y consumir servicios web basados en SOAP. Es una especificación de Jakarta EE que facilita el desarrollo de servicios web interoperables utilizando el protocolo SOAP y mensajes XML.

1.6. ¿Qué es JAX-RS?

JAX-RS es una API de Java utilizada para crear servicios web RESTful mediante HTTP. Es una especificación de Jakarta EE para la implementación de servicios web RESTful utilizando recursos identificados mediante URLs y operaciones HTTP.

1.7. ¿Qué es SOAP?

SOAP (**Simple Object Access Protocol**) es un protocolo de comunicación utilizado para intercambiar información entre aplicaciones mediante mensajes XML. SOAP utiliza mensajes XML transportados sobre HTTP.

1.8. ¿Qué es WSDL?

WSDL (**Web Services Description Language**) es un documento XML que describe las operaciones, parámetros y forma de comunicación de un servicio web SOAP.

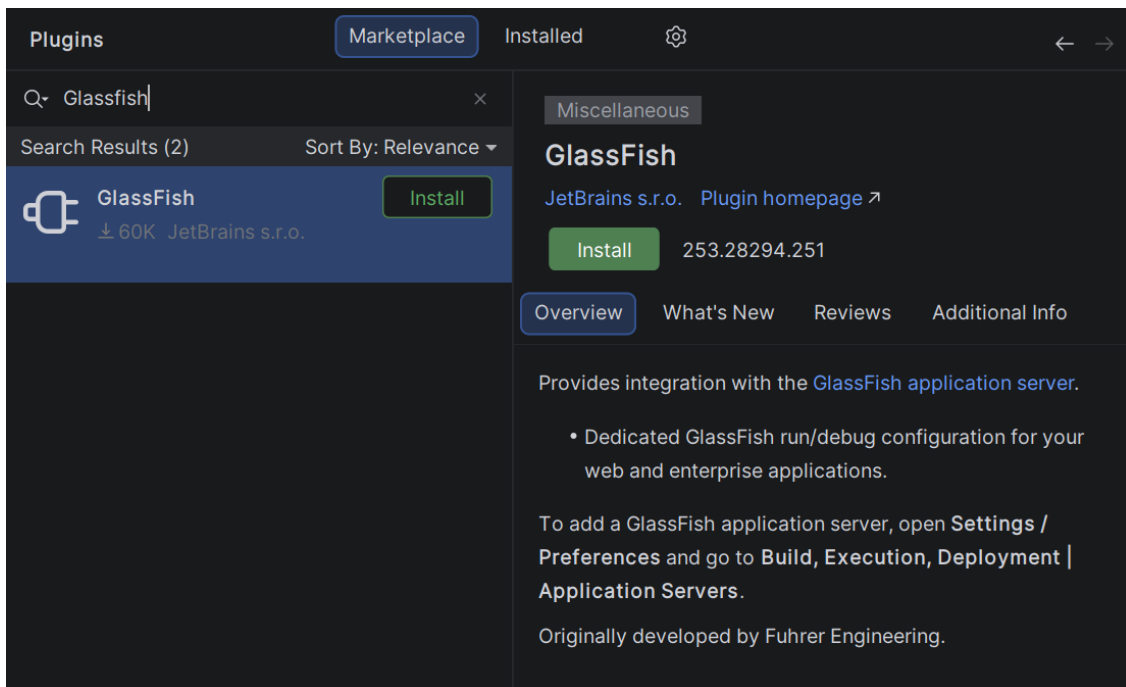
2. Configurando Glassfish en IntelliJ

2.1. Abrimos el IDE e ingresamos a la siguiente ruta:

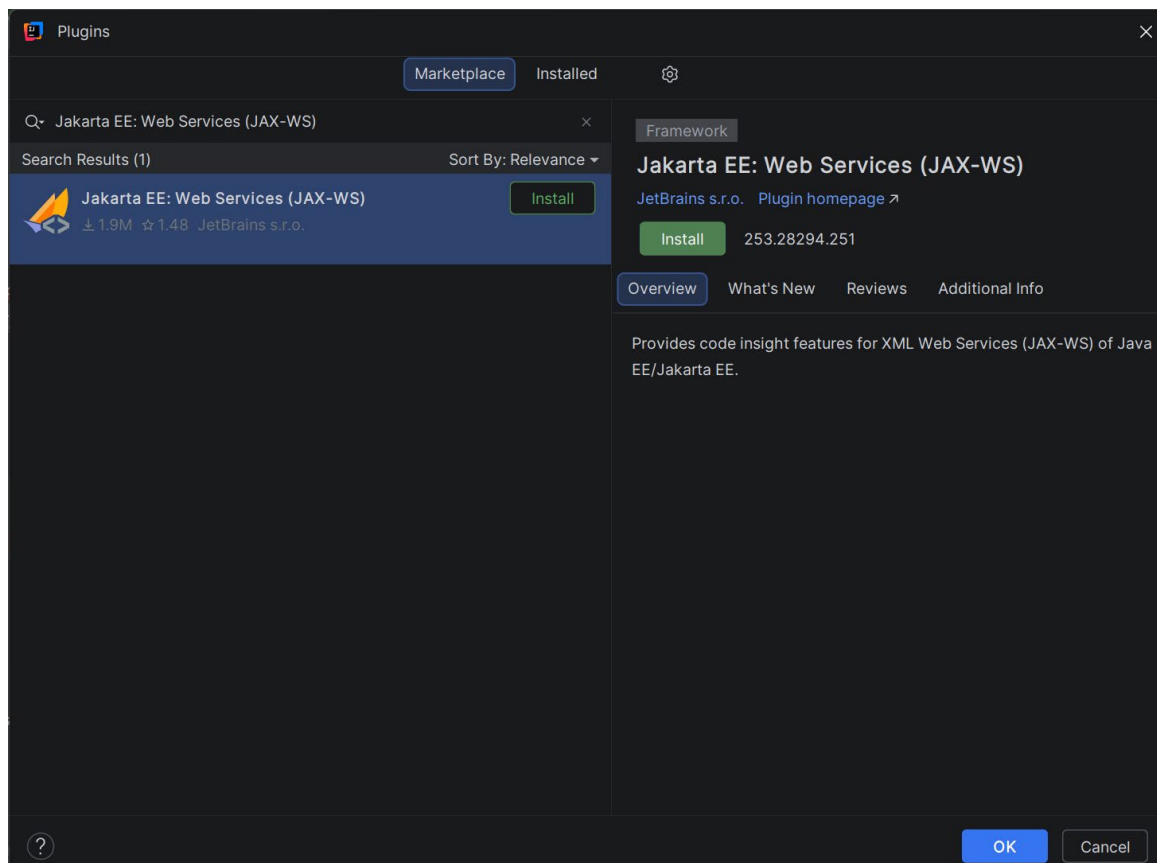
File > Settings > Plugins

Esto permitirá instalar un plugin para trabajar con GlassFish.

Instalamos el plugin “GlassFish”.

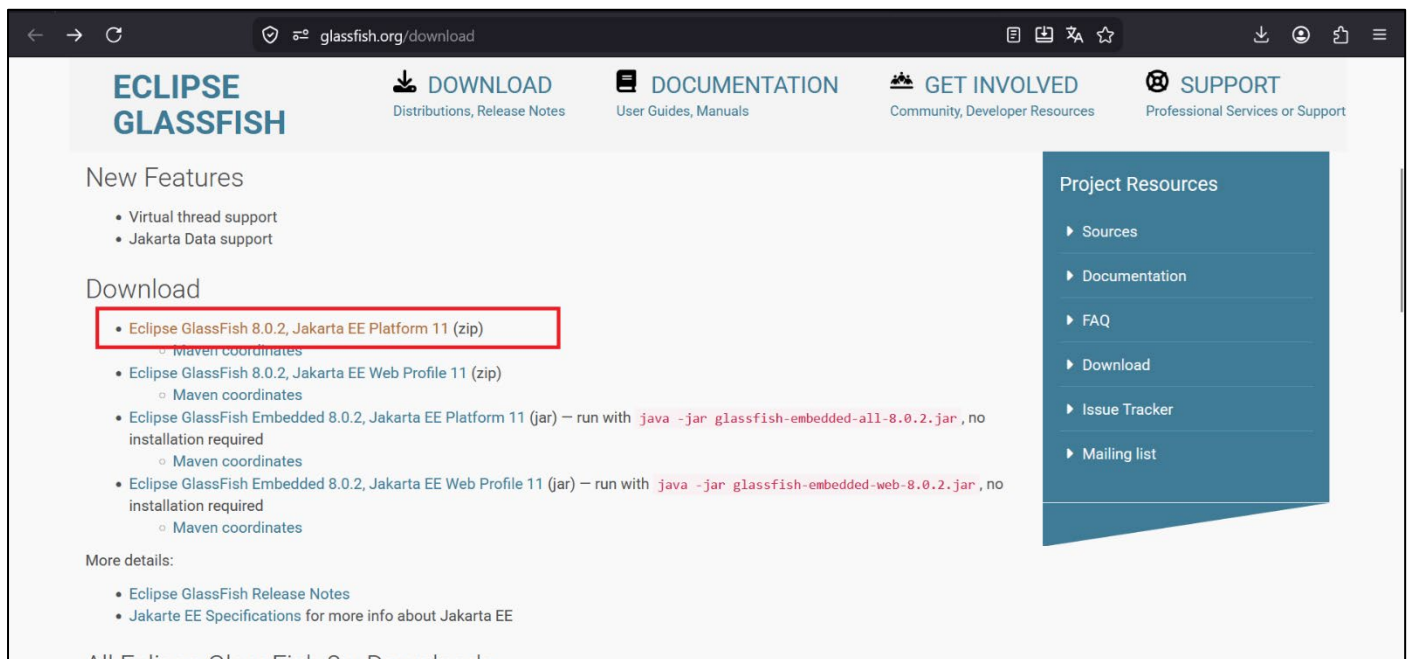


2.2. Instalamos también el plugin de Jakarta EE: Web Services (JAX-WS)



2.3. Ingresamos a la página oficial de Eclipse Glassfish y descargamos la última versión.

<https://glassfish.org/download>



The screenshot shows the Eclipse Glassfish download page. The navigation bar includes links for DOWNLOAD, DOCUMENTATION, GET INVOLVED, and SUPPORT. The main content area features a 'New Features' section, a 'Download' section with a list of download options, and a 'Project Resources' sidebar. The 'Download' section lists several options, with 'Eclipse GlassFish 8.0.2, Jakarta EE Platform 11 (zip)' highlighted by a red box. Below this, there are links for 'More details' and 'All Eclipse GlassFish 8.0 Downloads'.

ECLIPSE GLASSFISH

DOWNLOAD
Distributions, Release Notes

DOCUMENTATION
User Guides, Manuals

GET INVOLVED
Community, Developer Resources

SUPPORT
Professional Services or Support

New Features

- Virtual thread support
- Jakarta Data support

Download

- Eclipse GlassFish 8.0.2, Jakarta EE Platform 11 (zip)**
 - Maven coordinates
- Eclipse GlassFish 8.0.2, Jakarta EE Web Profile 11 (zip)
 - Maven coordinates
- Eclipse GlassFish Embedded 8.0.2, Jakarta EE Platform 11 (jar) — run with `java -jar glassfish-embedded-all-8.0.2.jar`, no installation required
 - Maven coordinates
- Eclipse GlassFish Embedded 8.0.2, Jakarta EE Web Profile 11 (jar) — run with `java -jar glassfish-embedded-web-8.0.2.jar`, no installation required
 - Maven coordinates

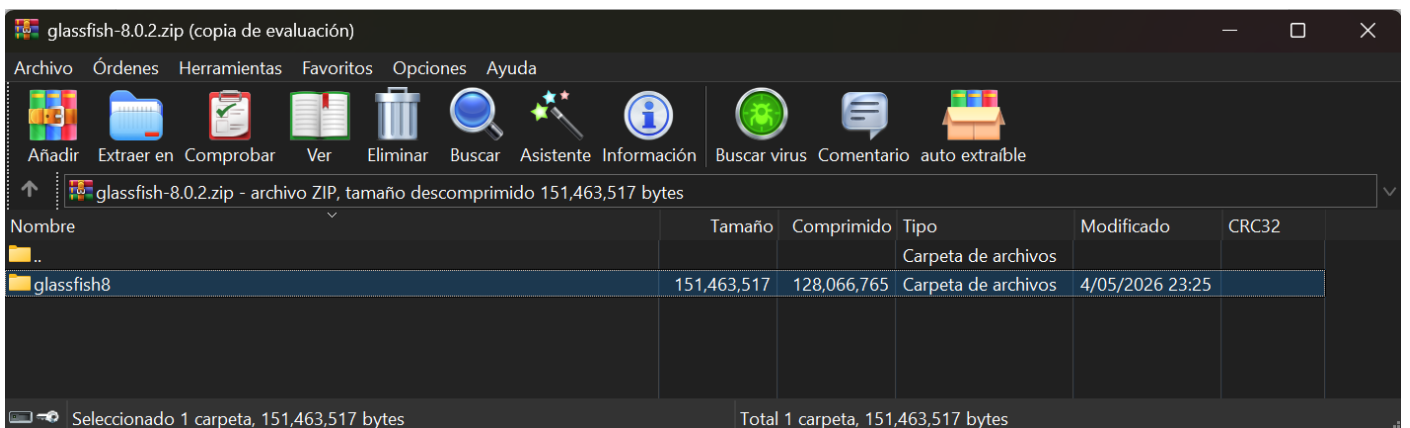
More details:

- Eclipse GlassFish Release Notes
- Jakarta EE Specifications for more info about Jakarta EE

Project Resources

- Sources
- Documentation
- FAQ
- Download
- Issue Tracker
- Mailing list

2.4. Descomprimos el .zip en alguna ubicación del computador



The screenshot shows a Windows File Explorer window titled 'glassfish-8.0.2.zip (copia de evaluación)'. The window displays the contents of the ZIP file, which is a folder named 'glassfish8'. The folder's size is 151,463,517 bytes, and it was modified on 4/05/2026 at 23:25. The window also shows the standard Windows File Explorer interface with a menu bar, a toolbar, and a status bar.

glassfish-8.0.2.zip (copia de evaluación)

Archivo Órdenes Herramientas Favoritos Opciones Ayuda

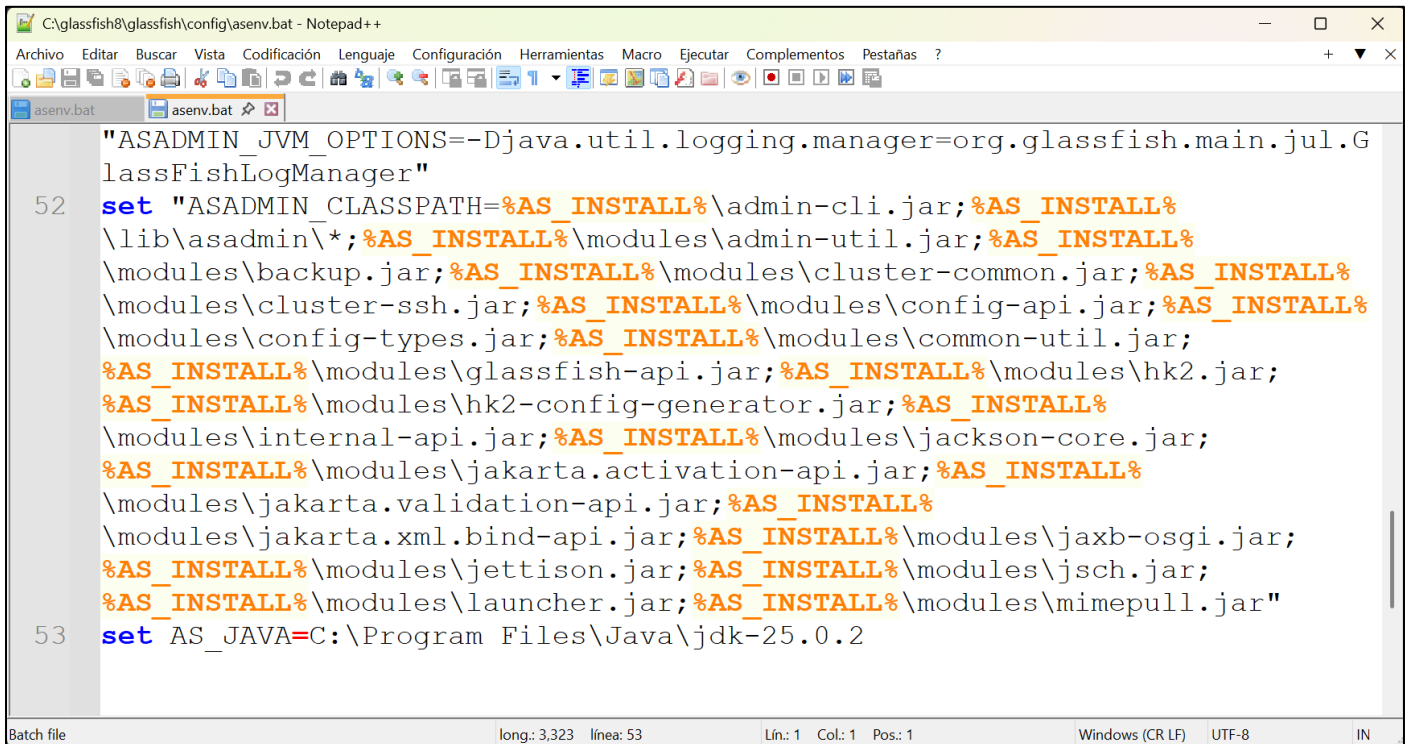
Añadir Extraer en Comprobar Ver Eliminar Buscar Asistente Información Buscar virus Comentario auto extraíble

↑ glassfish-8.0.2.zip - archivo ZIP, tamaño descomprimido 151,463,517 bytes

Nombre	Tamaño	Comprimido	Tipo	Modificado	CRC32
..			Carpeta de archivos		
glassfish8	151,463,517	128,066,765	Carpeta de archivos	4/05/2026 23:25	

Seleccionado 1 carpeta, 151,463,517 bytes Total 1 carpeta, 151,463,517 bytes

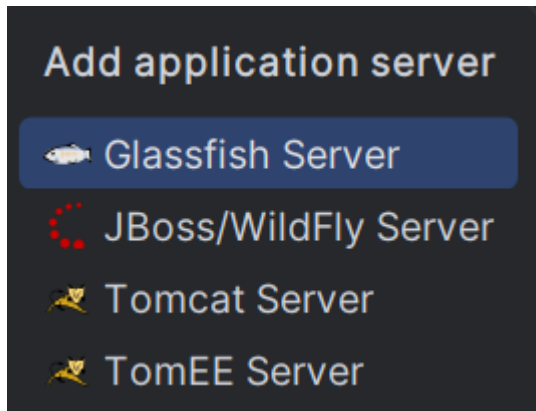
- 2.5. Donde se haya descargado el "glassfish", ingresamos a la siguiente ruta:
<ruta>\glassfish\config y ubicamos el archivo "asenv.bat".
Abrimos ese archivo con un Notepad++ y agregamos la siguiente línea
set AS_JAVA=<ruta donde está instalado el JDK>
Debe quedar de la siguiente manera:



```
C:\glassfish8\glassfish\config\asenv.bat - Notepad++
Archivo  Editar  Buscar  Vista  Codificación  Lenguaje  Configuración  Herramientas  Macro  Ejecutar  Complementos  Pestañas  ?
asenv.bat  asenv.bat
"ASADMIN_JVM_OPTIONS=-Djava.util.logging.manager=org.glassfish.main.jul.GlassFishLogManager"
52  set "ASADMIN_CLASSPATH=%AS_INSTALL%\admin-cli.jar;%AS_INSTALL%\lib\asadmin\*; %AS_INSTALL%\modules\admin-util.jar;%AS_INSTALL%\modules\backup.jar;%AS_INSTALL%\modules\cluster-common.jar;%AS_INSTALL%\modules\cluster-ssh.jar;%AS_INSTALL%\modules\config-api.jar;%AS_INSTALL%\modules\config-types.jar;%AS_INSTALL%\modules\common-util.jar;%AS_INSTALL%\modules\glassfish-api.jar;%AS_INSTALL%\modules\hk2.jar;%AS_INSTALL%\modules\hk2-config-generator.jar;%AS_INSTALL%\modules\internal-api.jar;%AS_INSTALL%\modules\jackson-core.jar;%AS_INSTALL%\modules\jakarta.activation-api.jar;%AS_INSTALL%\modules\jakarta.validation-api.jar;%AS_INSTALL%\modules\jakarta.xml.bind-api.jar;%AS_INSTALL%\modules\jaxb-osgi.jar;%AS_INSTALL%\modules\jettison.jar;%AS_INSTALL%\modules\jsch.jar;%AS_INSTALL%\modules\launcher.jar;%AS_INSTALL%\modules\mimepull.jar"
53  set AS_JAVA=C:\Program Files\Java\jdk-25.0.2
Batch file  long.: 3,323  línea: 53  Lín.: 1  Col.: 1  Pos.: 1  Windows (CR LF)  UTF-8  IN
```

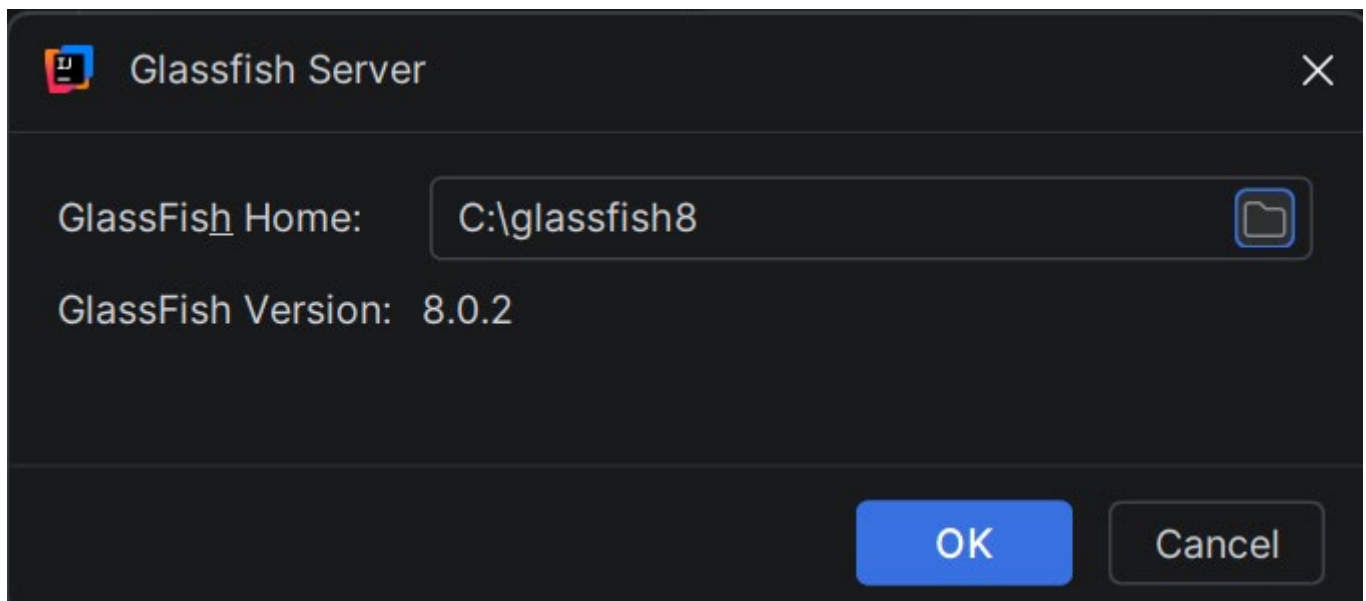
- 2.6. Guardamos y cerramos.

- 2.7. Ingresamos al IDE a la siguiente ruta:
File > Settings > Build, Execution, Deployment > Application Servers > +



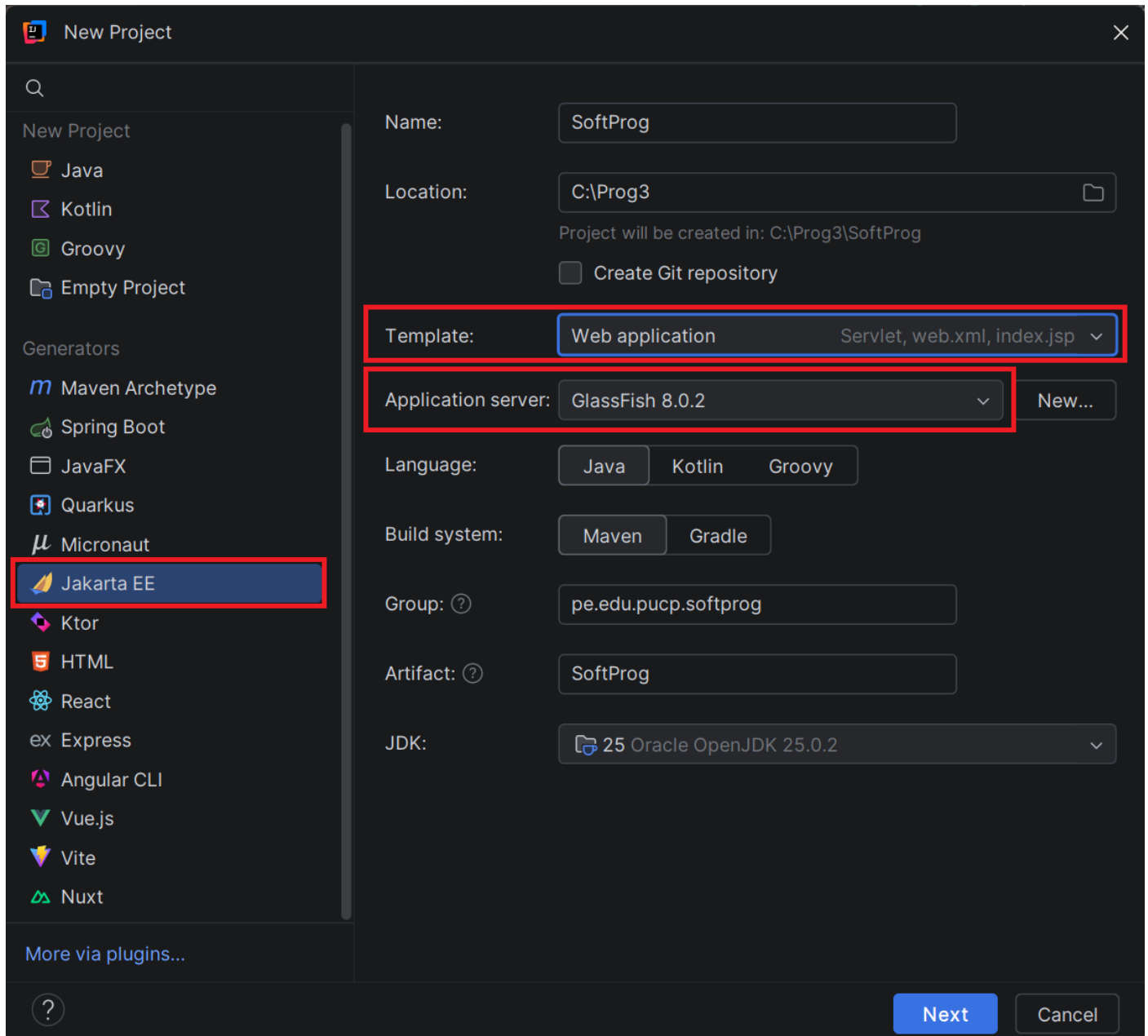
- 2.8. Elegimos "Glassfish Server".

- 2.9. Indicamos la ruta (carpeta) donde fue descomprimido el Glassfish Server. Damos clic en OK.



3. Creación de Servicios SOAP

3.1. Creamos un nuevo proyecto de tipo “Jakarta EE” con template de “Web Application”.



New Project

Q

New Project

- Java
- Kotlin
- Groovy
- Empty Project

Generators

- Maven Archetype
- Spring Boot
- JavaFX
- Quarkus
- Micronaut
- Jakarta EE**
- Ktor
- HTML
- React
- Express
- Angular CLI
- Vue.js
- Vite
- Nuxt

More via plugins...

Name: SoftProg

Location: C:\Prog3
Project will be created in: C:\Prog3\SoftProg
☐ Create Git repository

Template: Web application Servlet, web.xml, index.jsp

Application server: GlassFish 8.0.2 New...

Language: Java Kotlin Groovy

Build system: Maven Gradle

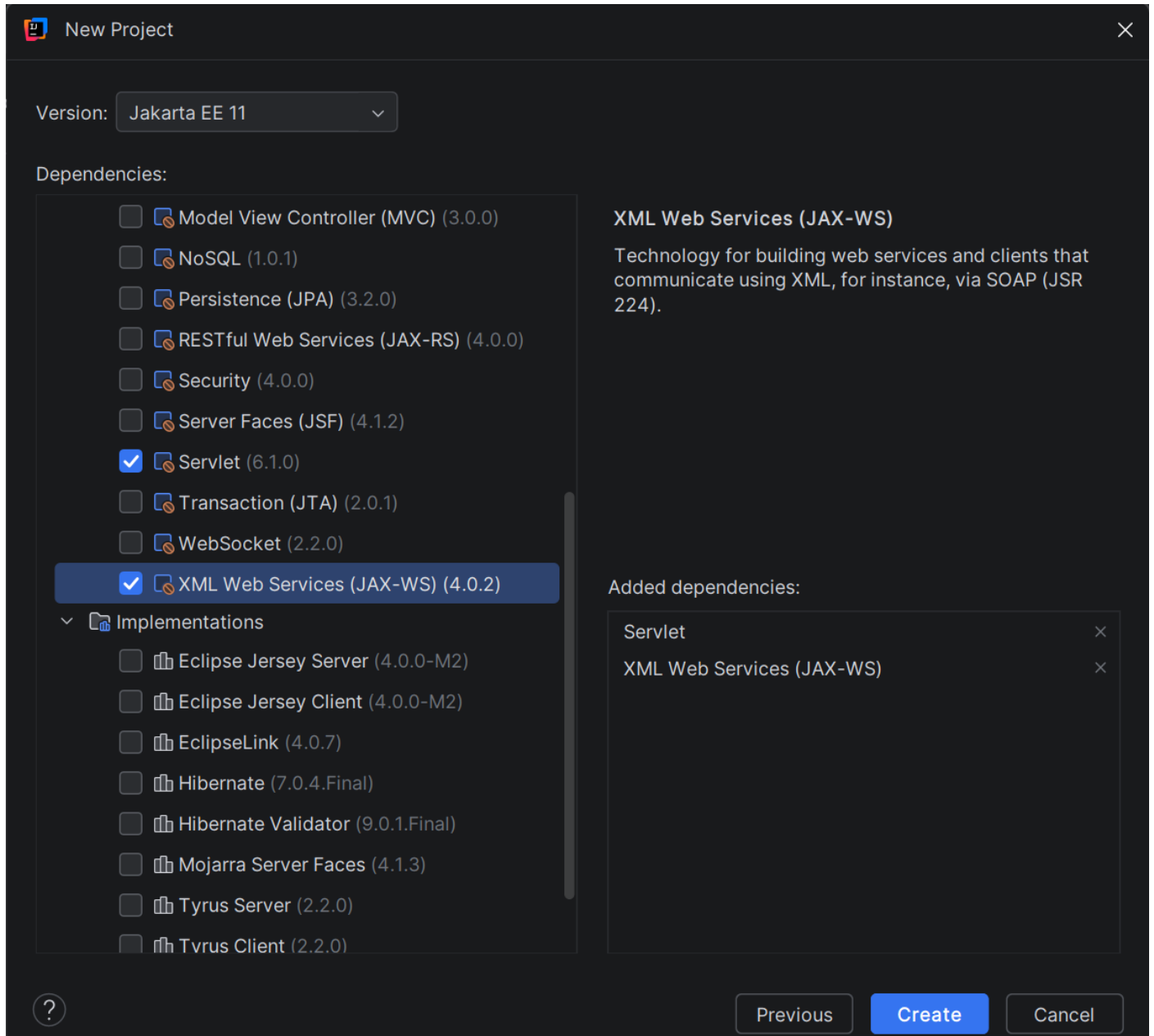
Group: ? pe.edu.pucp.softprog

Artifact: ? SoftProg

JDK: 25 Oracle OpenJDK 25.0.2

Next Cancel

3.2. En las dependencias, seleccionamos **"XML Web Services (JAX-WS)"** para agregar las librerías necesarias que permitirán desarrollar servicios web SOAP. Adicionalmente, es posible seleccionar la dependencia **"Servlet"**, ya que la plantilla generará por defecto una página basada en esta tecnología, evitando así posibles errores de compilación o referencias faltantes.



New Project

Version: Jakarta EE 11

Dependencies:

- ☐ Model View Controller (MVC) (3.0.0)
- ☐ NoSQL (1.0.1)
- ☐ Persistence (JPA) (3.2.0)
- ☐ RESTful Web Services (JAX-RS) (4.0.0)
- ☐ Security (4.0.0)
- ☐ Server Faces (JSF) (4.1.2)
- ☒ Servlet (6.1.0)
- ☐ Transaction (JTA) (2.0.1)
- ☐ WebSocket (2.2.0)
- ☒ XML Web Services (JAX-WS) (4.0.2)

▼ Implementations

- ☐ Eclipse Jersey Server (4.0.0-M2)
- ☐ Eclipse Jersey Client (4.0.0-M2)
- ☐ EclipseLink (4.0.7)
- ☐ Hibernate (7.0.4.Final)
- ☐ Hibernate Validator (9.0.1.Final)
- ☐ Mojarra Server Faces (4.1.3)
- ☐ Tyrus Server (2.2.0)
- ☐ Tyrus Client (2.2.0)

XML Web Services (JAX-WS)

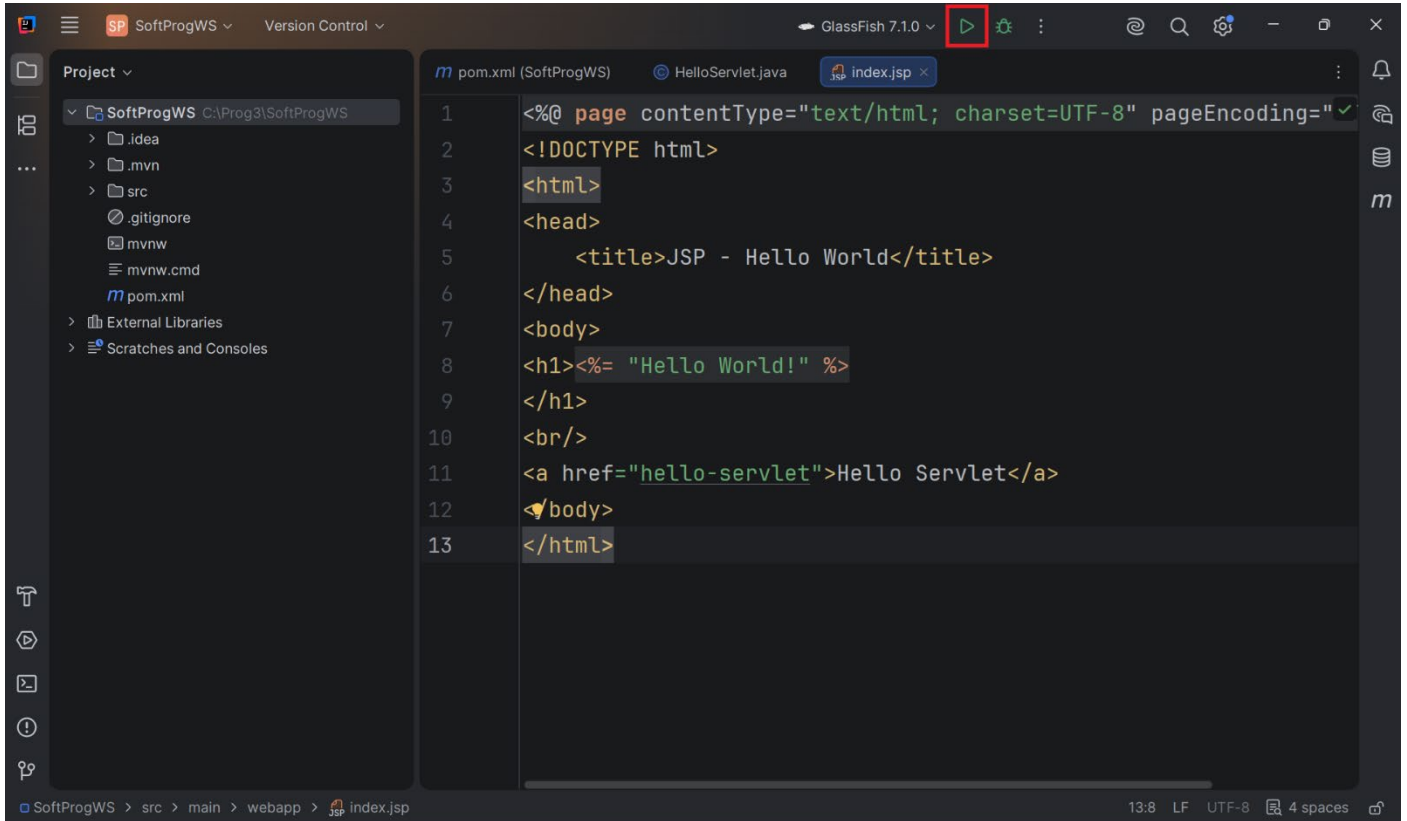
Technology for building web services and clients that communicate using XML, for instance, via SOAP (JSR 224).

Added dependencies:

- Servlet
- XML Web Services (JAX-WS)

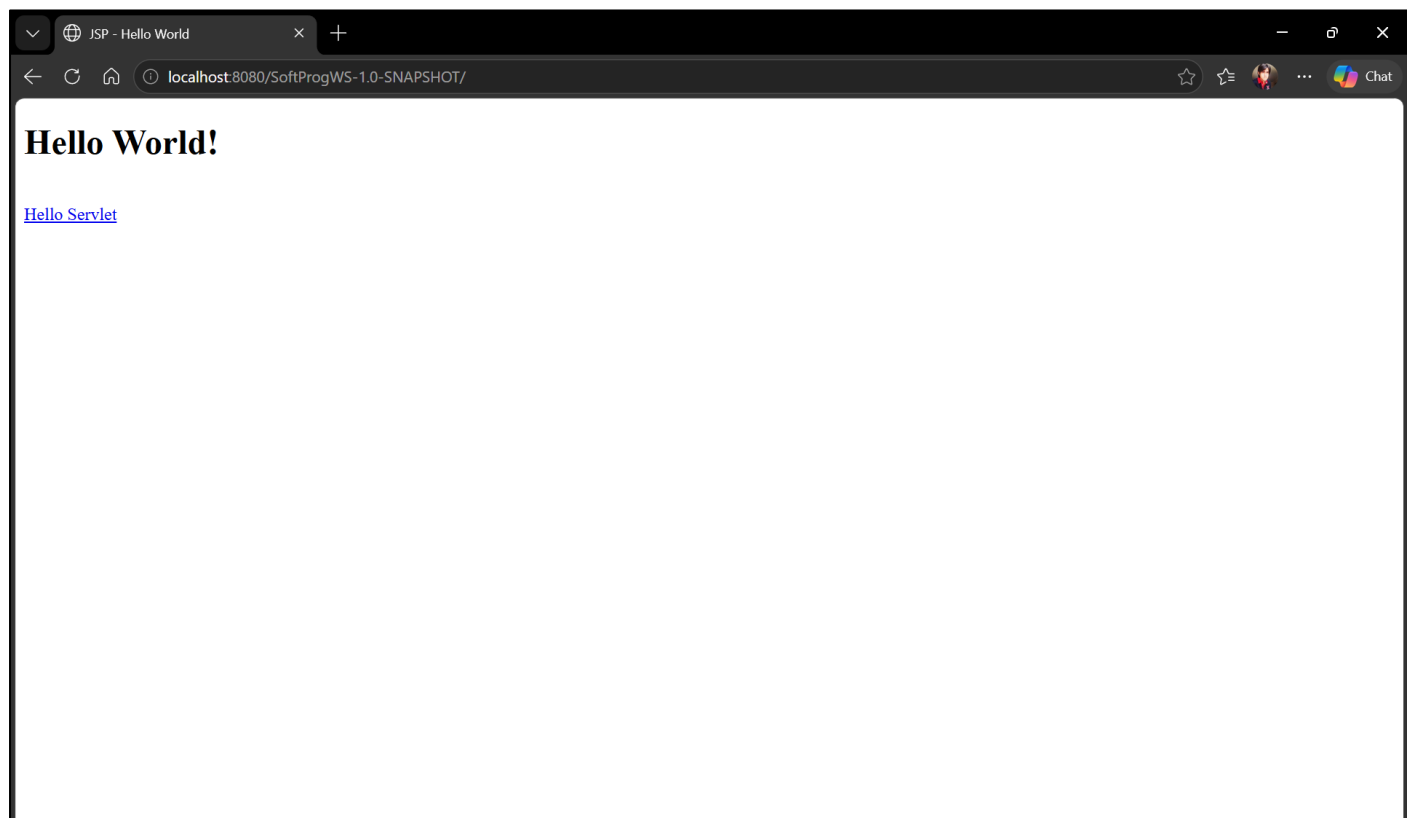
Previous Create Cancel

3.3. Una vez creado el proyecto web, le damos al botón “Run”



```
1 <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
2 <!DOCTYPE html>
3 <html>
4 <head>
5     <title>JSP - Hello World</title>
6 </head>
7 <body>
8     <h1><%= "Hello World!" %>
9 </h1>
10    <br/>
11    <a href="hello-servlet">Hello Servlet</a>
12 </body>
13 </html>
```

3.4. Se abrirá la página web del proyecto.



- 3.5. Procedemos a crear un servicio SOAP
Creamos un paquete "pe.edu.pucp.softprog.services"
Creamos una clase "PruebaWS"

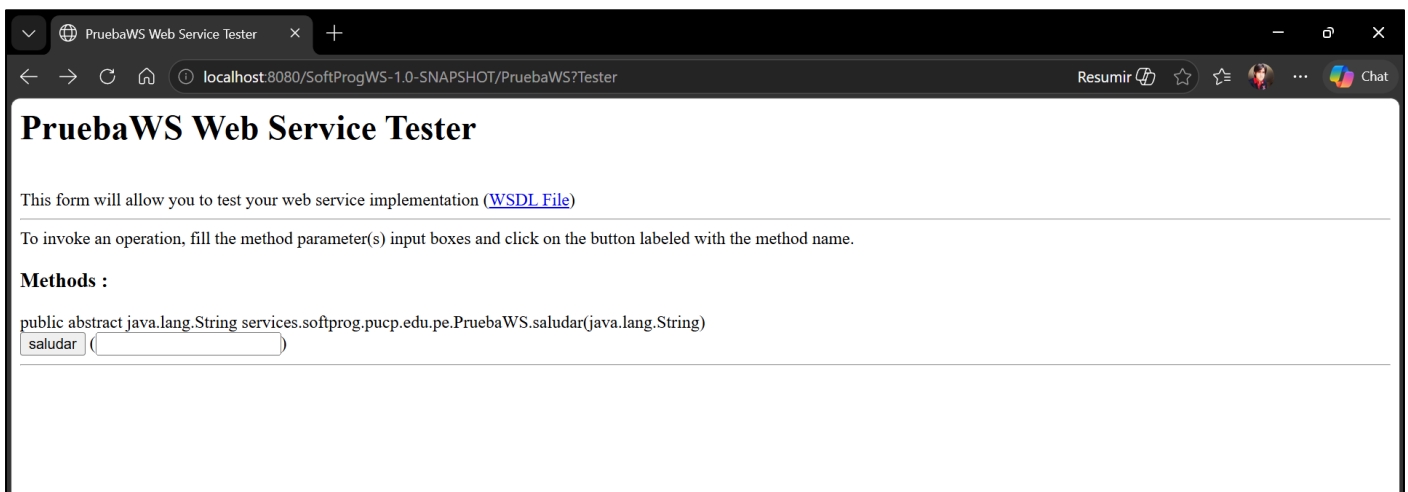
```
package pe.edu.pucp.softprog.services;  
  
import jakarta.jws.WebMethod;  
import jakarta.jws.WebService;  
  
@WebService(  
    serviceName = "PruebaWS",  
    targetNamespace = "http://services.softprog.pucp.edu.pe/"  
)  
public class PruebaWS {  
  
    @WebMethod(operationName = "saludar")  
    public String saludar(String nombre) {  
        return "Hola " + nombre;  
    }  
}
```

- 3.6. Volvemos a ejecutar el proyecto
Una vez desplegado el servicio web en GlassFish, el servidor genera automáticamente una página web de prueba que permite visualizar y probar las operaciones disponibles del servicio. Esta página puede accederse agregando ?Tester al final de la URL del servicio.

Asimismo, la descripción WSDL del servicio puede consultarse agregando ?wsdl al final de la URL del servicio.

Por ejemplo:

<http://localhost:8080/SoftProgWS-1.0-SNAPSHOT/PruebaWS?Tester>
<http://localhost:8080/SoftProgWS-1.0-SNAPSHOT/PruebaWS?WSDL>



localhost:8080/SoftProgWS-1.0-SNAPSHOT/PruebaWS?WSDL

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<!-- Published by XML-WS Runtime (https://github.com/eclipse-ee4j/metro-jax-ws). Runtime's version is Eclipse Metro/4.0.4 (RELEASE-4.0.4-ec440d5; 2024-06-28T10:09:00+0000)
<!-- Generated by XML-WS Runtime (https://github.com/eclipse-ee4j/metro-jax-ws). Runtime's version is Eclipse Metro/4.0.4 (RELEASE-4.0.4-ec440d5; 2024-06-28T10:09:00+0000)
<definitions xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" xmlns:wsp="http://www.w3.org/ns/ws-policy"
xmlns:wsp1_2="http://schemas.xmlsoap.org/ws/2004/09/policy" xmlns:wsam="http://www.w3.org/2007/05/addressing/metadata" xmlns:soap="http://schemas.xmlsoap.org/soap/"
xmlns:tns="pe.edu.pucp.softprog.services" xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://schemas.xmlsoap.org/wsdl/"
targetNamespace="pe.edu.pucp.softprog.services" name="PruebaWS">
  <types>
    <xsd:schema>
      <xsd:import namespace="pe.edu.pucp.softprog.services" schemaLocation="http://localhost:8080/SoftProgWS-1.0-SNAPSHOT/PruebaWS?xsd=1"/>
    </xsd:schema>
  </types>
  <message name="saludar">
    <part name="parameters" element="tns:saludar"/>
  </message>
  <message name="saludarResponse">
    <part name="parameters" element="tns:saludarResponse"/>
  </message>
  <portType name="PruebaWS">
    <operation name="saludar">
      <input wsam:Action="pe.edu.pucp.softprog.services/PruebaWS/saludarRequest" message="tns:saludar"/>
      <output wsam:Action="pe.edu.pucp.softprog.services/PruebaWS/saludarResponse" message="tns:saludarResponse"/>
    </operation>
  </portType>
  <binding name="PruebaWSPortBinding" type="tns:PruebaWS">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http" style="document"/>
    <operation name="saludar">
      <soap:operation soapAction=""/>
      <input>
        <soap:body use="literal"/>
      </input>
      <output>
        <soap:body use="literal"/>
      </output>
    </operation>
  </binding>
  <service name="PruebaWS">
    <port name="PruebaWSPort" binding="tns:PruebaWSPortBinding">
      <soap:address location="http://localhost:8080/SoftProgWS-1.0-SNAPSHOT/PruebaWS"/>
    </port>
  </service>
</definitions>
```

3.7. Probamos el servicio vía web

Method invocation trace

localhost:8080/SoftProgWS-1.0-SNAPSHOT/PruebaWS?Tester

saludar Method invocation

Method parameter(s)

Type	Value
java.lang.String	Juan Perez

Method returned

java.lang.String : "Hola Juan Perez"

SOAP Request

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body>
    <ns2:saludar xmlns:ns2="pe.edu.pucp.softprog.services">
      <arg0>Juan Perez</arg0>
    </ns2:saludar>
  </S:Body>
</S:Envelope>
```

SOAP Response

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body>
    <ns2:saludarResponse xmlns:ns2="pe.edu.pucp.softprog.services">
      <return>Hola Juan Perez</return>
    </ns2:saludarResponse>
  </S:Body>
</S:Envelope>
```

Etiquetas Utilizadas:

Etiqueta	Descripción
@WebService	Indica que la clase será publicada como un servicio web SOAP.
serviceName	Define el nombre del servicio web que aparecerá en el WSDL.
targetNamespace	Define el espacio de nombres XML utilizado por el servicio web.
@WebMethod	Indica que un método será expuesto como una operación del servicio web.
operationName	Define el nombre de la operación SOAP expuesta en el servicio.
@WebParam	Define el nombre y configuración de un parámetro de entrada de una operación SOAP.

3.8. En una clase distinta, creamos un servicio que ofrezca los métodos “sumar”, “restar”, “multiplicar” y “dividir”.

```
package pe.edu.pucp.softprog.services;
import jakarta.jws.WebMethod;
import jakarta.jws.WebParam;
import jakarta.jws.WebService;

@WebService(
    serviceName = "CalculadoraWS",
    targetNamespace = "http://services.softprog.pucp.edu.pe/"
)
public class CalculadoraWS {

    @WebMethod(operationName = "sumar")
    public double sumar(
        @WebParam(name = "numero1") double numero1,
        @WebParam(name = "numero2") double numero2) {

        return numero1 + numero2;
    }

    @WebMethod(operationName = "restar")
    public double restar(
        @WebParam(name = "numero1") double numero1,
        @WebParam(name = "numero2") double numero2) {
        return numero1 - numero2;
    }

    @WebMethod(operationName = "multiplicar")
    public double multiplicar(
        @WebParam(name = "numero1") double numero1,
        @WebParam(name = "numero2") double numero2) {

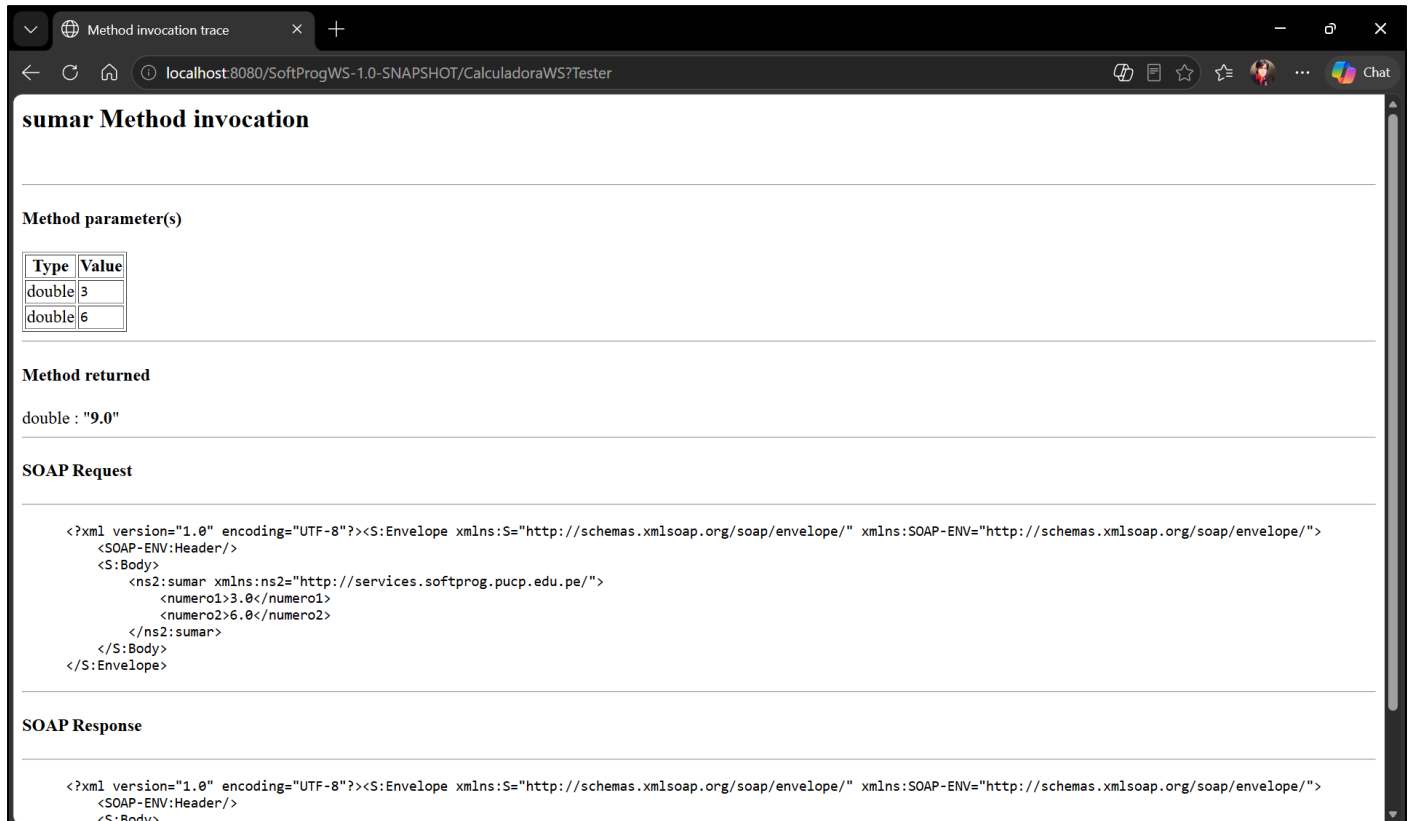
        return numero1 * numero2;
    }

    @WebMethod(operationName = "dividir")
    public double dividir(
        @WebParam(name = "numero1") double numero1,
        @WebParam(name = "numero2") double numero2) {

        if (numero2 == 0) {
            throw new ArithmeticException(
                "No es posible dividir entre cero.");
        }

        return numero1 / numero2;
    }
}
```

3.9. Verificamos que los métodos funcionen



Method invocation trace

localhost:8080/SoftProgWS-1.0-SNAPSHOT/CalculadoraWS?Tester

sumar Method invocation

Method parameter(s)

Type	Value
double	3
double	6

Method returned

double : "9.0"

SOAP Request

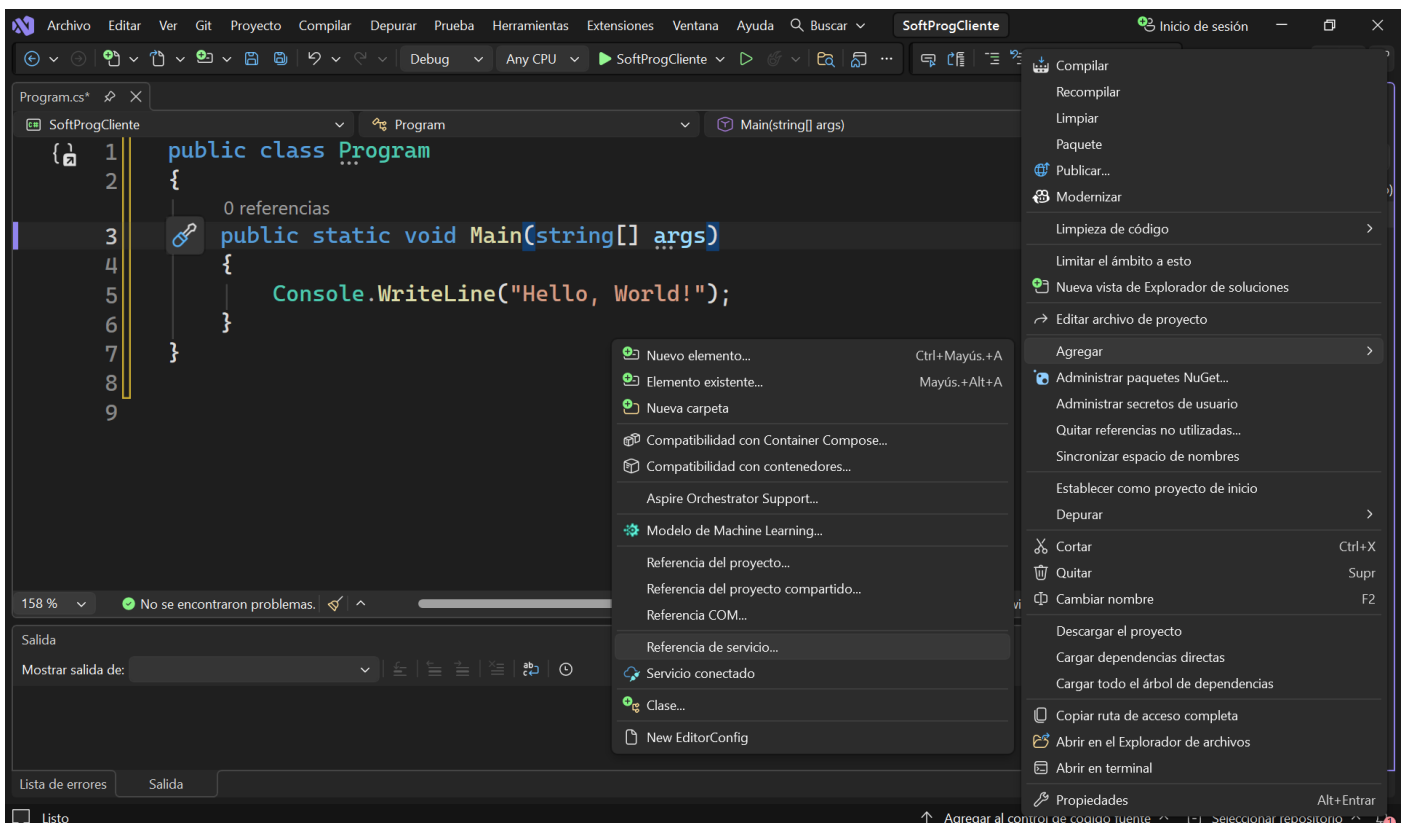
```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body>
    <ns2:sumar xmlns:ns2="http://services.softprog.pucp.edu.pe/">
      <numero1>3.0</numero1>
      <numero2>6.0</numero2>
    </ns2:sumar>
  </S:Body>
</S:Envelope>
```

SOAP Response

```
<?xml version="1.0" encoding="UTF-8"?><S:Envelope xmlns:S="http://schemas.xmlsoap.org/soap/envelope/" xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP-ENV:Header/>
  <S:Body>
```

4. Consumimos los servicios desde C#

4.1. En un proyecto en consola en C#, hacemos clic derecho sobre el proyecto,
Agregar -> Referencia de servicio...



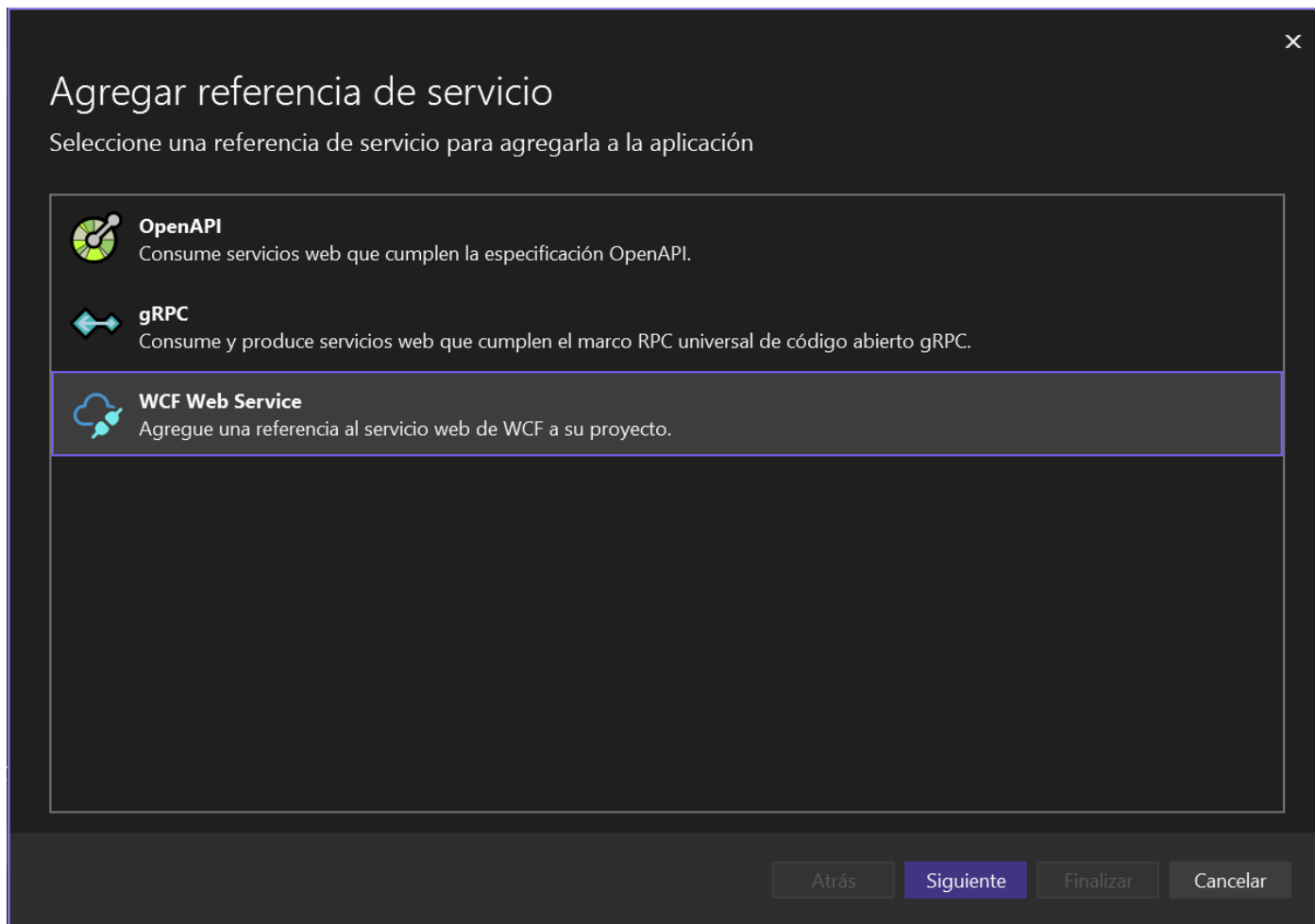
Visual Studio interface showing a C# console application named 'SoftProgCliente'.

The code in Program.cs is:

```
1 public class Program
2 {
3     public static void Main(string[] args)
4     {
5         Console.WriteLine("Hello, World!");
6     }
7 }
8
9
```

The right-click context menu is open, showing options like 'Agregar', 'Referencia de servicio...', 'Servicio conectado', etc.

4.2. Elegimos WCF Web Service



4.3. Colocamos la dirección WSDL del servicio y establecemos un Espacio de nombres en nuestro proyecto C#.

×

Agregar una nueva referencia de servicio WCF Web Service

Especifique el servicio para agregar

Para ver una lista de los servicios disponibles en un servidor determinado, escriba una dirección URL de servicio y haga clic en Ir. Para buscar los servicios disponibles en la solución, haga clic en Detectar. Para cargar metadatos de un servicio desde un archivo WSDL, haga clic en Examinar.

URI:

http://localhost:8080/SoftProgWS-1.0-SNAPSHOT/CalculadoraWS?WSDL

DetenerDetectarExaminar...

Servicios:

Operaciones:

CalculadoraWS

CalculadoraWS

sumar

restar

dividir

multiplicar

Estado:

Número de servicios encontrados: 1

Espacio de nombres:

ServiciosWeb

AtrásSiguienteFinalizarCancelar

El espacio de nombres definido al agregar la referencia del servicio en C# permitirá acceder a las clases proxy generadas automáticamente por Visual Studio. Gracias a ello, será posible instanciar objetos del servicio web y consumir las operaciones implementadas en Java. Se recomienda que el espacio de nombres (namespace) definido al agregar la referencia del servicio en C# sea diferente al nombre del servicio web SOAP. Utilizar el mismo nombre puede generar conflictos de nombres en las clases proxy generadas automáticamente por Visual Studio.

4.4. Establecemos el tipo de colección para que las colecciones devueltas por el servicio Java sean representadas como listas genéricas en C#.

×

Agregar una nueva referencia de servicio WCF Web Service

Especificar las opciones de tipo de datos

☐ Generar siempre contratos de mensaje

Tipo de colección:

System.Collections.Generic.List

Tipo de colección de diccionario:

System.Collections.Generic.Dictionary

☒ Reutilizar tipos en los ensamblados a los que se hace referencia

- ☐ Reutilizar tipos en todos los ensamblados a los que se hace referencia
- ☐ Reutilizar tipos en los ensamblados especificados:

Atrás

Siguiente

Finalizar

Cancelar

4.5. Se recomienda mantener el nivel de acceso de las clases generadas como Public, de manera que las clases proxy creadas automáticamente puedan ser utilizadas desde cualquier parte del proyecto C#.

×

Agregar una nueva referencia de servicio WCF Web Service

Especificar las opciones de cliente

Nivel de acceso de las clases generadas:

☐ Public

☒ Internal

☐ Habilitar enlace de datos

Modo de generación de la Operación:

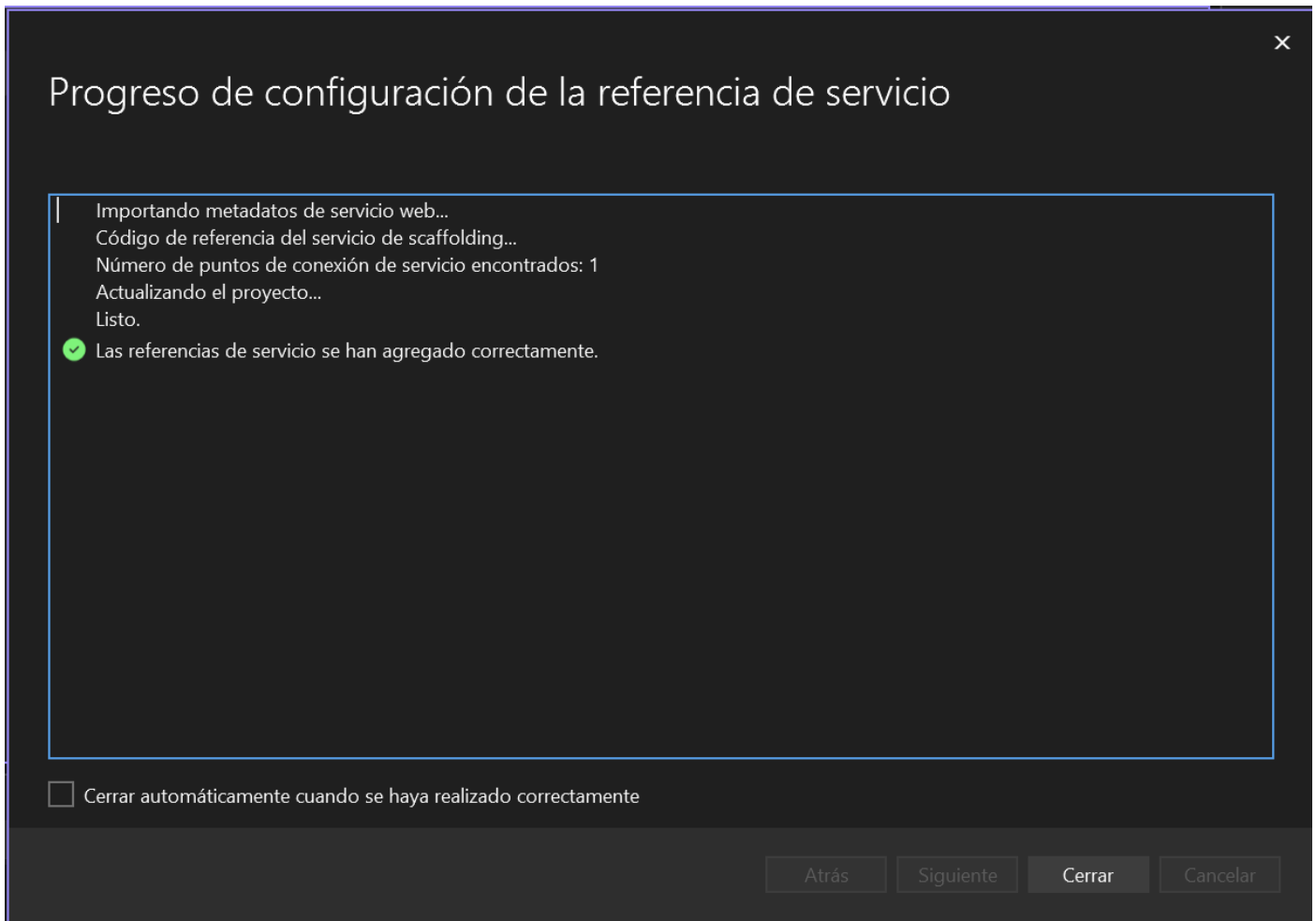
Atrás

Siguiente

Finalizar

Cancelar

4.6. Cerramos



4.7. Programamos la llamada a los servicios

Una vez agregada la referencia del servicio WSDL, Visual Studio generará automáticamente las clases proxy necesarias para consumir las operaciones implementadas en Java. En el siguiente ejemplo se instancia un objeto del servicio web y se invocan las operaciones sumar, restar, multiplicar y dividir.

```
using ServiciosWeb;

public class Program
{
    public static void Main(string[] args)
    {
        CalculadoraWSDLClient ws = new CalculadoraWSDLClient();

        double suma = ws.sumar(10, 5);
        double resta = ws.restar(10, 5);
        double multiplicacion = ws.multiplicar(10, 5);
        double division = ws.dividir(10, 5);

        Console.WriteLine("Suma: " + suma);
        Console.WriteLine("Resta: " + resta);
        Console.WriteLine("Multiplicación: " + multiplicacion);
        Console.WriteLine("División: " + division);
    }
}
```

```
Consola de depuración de Mi x + v
Suma: 15
Resta: 5
Multiplicación: 50
División: 2

C:\Prog3\ConsoleApp1\ConsoleApp1\ConsoleApp1\bin\Debug\net10.0\ConsoleApp1.exe (proceso 22420
) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas
->Opciones ->Depuración ->Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .|
```